

torch.functional.lu#

<https://pytorch.org/docs/stable/generated/torch.functional.lu.html>

Description:

Computes the LU factorization of a matrix or batches of matrices A. Returns a tuple containing the LU factorization and pivots of A. Pivoting is done if pivot is set to True. `torch.lu()` is deprecated in favor of `torch.linalg.lu_factor()` and `torch.linalg.lu_factor_ex()`. `torch.lu()` will be removed in a future PyTorch release. `LU, pivots, info = torch.lu(A, compute_pivots)` should be replaced with `LU, pivots, info = torch.lu(A, compute_pivots, get_infos=True)` should be replaced with

Function Signature

```
torch.functional.lu(*args, **kwargs)[source]#
```

Parameters

Returns

Return type

Returns:

(Tensor, IntTensor, IntTensor (optional))

Code Examples

```
LU, pivots = torch.linalg.lu_factor(A, compute_pivots)
```

```
LU, pivots, info = torch.linalg.lu_factor_ex(A, compute_pivots)
```

```
>>> A = torch.randn(2, 3, 3)
>>> A_LU, pivots = torch.lu(A)
>>> A_LU
tensor([[[[ 1.3506,  2.5558, -0.0816],
           [ 0.1684,  1.1551,  0.1940],
           [ 0.1193,  0.6189, -0.5497]],
          [[ 0.4526,  1.2526, -0.3285],
           [-0.7988,  0.7175, -0.9701],
           [ 0.2634, -0.9255, -0.3459]]]])
>>> pivots
tensor([[ 3,  3,  3],
        [ 3,  3,  3]], dtype=torch.int32)
>>> A_LU, pivots, info = torch.lu(A, get_infos=True)
>>> if info.nonzero().size(0) == 0:
...     print('LU factorization succeeded for all samples!')
LU factorization succeeded for all samples!
```

Notes & Warnings

⚠ WARNING

Warning `torch.lu()` is deprecated in favor of `torch.linalg.lu_factor()` and `torch.linalg.lu_factor_ex()`. `torch.lu()` will be removed in a future PyTorch release. `LU, pivots, info = torch.lu(A, compute_pivots)` should be replaced with `LU, pivots = torch.linalg.lu_factor(A, compute_pivots)` `LU, pivots, info = torch.lu(A, compute_pivots, get_infos=True)` should be replaced with `LU, pivots, info = torch.linalg.lu_factor_ex(A, compute_pivots)`

NOTE

Note The returned permutation matrix for every matrix in the batch is represented by a 1-indexed vector of size `min(A.shape[-2], A.shape[-1])`. `pivots[i] == j` represents that in the *i*-th step of the algorithm, the *i*-th row was permuted with the *j*-1-th row. LU factorization with `pivot = False` is not available for CPU, and attempting to do so will throw an error. However, LU factorization with `pivot = False` is available for CUDA. This function does not check if the factorization was successful or not if `get_infos` is `True` since the status of the factorization is present in the third element of the return tuple. In the case of batches of square matrices with size less or equal to 32 on a CUDA device, the LU factorization is repeated for singular matrices due to the bug in the MAGMA library (se

⚠ WARNING

Warning The gradients of this function will only be finite when A is full rank. This is because the LU decomposition is just differentiable at full rank matrices. Furthermore, if A is close to not being full rank, the gradient will be numerically unstable as it depends on the computation of $L^{-1}L^{-1}L^{-1}$ and $U^{-1}U^{-1}U^{-1}$.

Detailed Documentation

Documentation

Computes the LU factorization of a matrix or batches of matrices A. Returns a tuple containing the LU factorization and pivots of A. Pivoting is done if pivot is set to True.

`torch.lu()` is deprecated in favor of `torch.linalg.lu_factor()` and `torch.linalg.lu_factor_ex()`. `torch.lu()` will be removed in a future PyTorch release. `LU, pivots, info = torch.lu(A, compute_pivots)` should be replaced with

`LU, pivots, info = torch.lu(A, compute_pivots, get_infos=True)` should be replaced with

The returned permutation matrix for every matrix in the batch is represented by a 1-indexed vector of size `min(A.shape[-2], A.shape[-1])`. `pivots[i] == j` represents that in the *i*-th step of the algorithm, the *i*-th row was permuted with the *j*-1-th row.

LU factorization with `pivot = False` is not available for CPU, and attempting to do so will throw an error. However, LU factorization with `pivot = False` is available for CUDA.

This function does not check if the factorization was successful or not if `get_infos` is True since the status of the factorization is present in the third element of the return

tuple.

In the case of batches of square matrices with size less or equal to 32 on a CUDA device, the LU factorization is repeated for singular matrices due to the bug in the MAGMA library (see magma issue 13).

L, U, and P can be derived using `torch.lu_unpack()`.

The gradients of this function will only be finite when A is full rank. This is because the LU decomposition is just differentiable at full rank matrices. Furthermore, if A is close to not being full rank, the gradient will be numerically unstable as it depends on the computation of $L^{-1}L^{-1}L^{-1}$ and $U^{-1}U^{-1}U^{-1}$.

A (Tensor) – the tensor to factor of size $(*, m, n)(*, m, n)(*, m, n)$

pivot (bool, optional) – Whether to compute the LU decomposition with partial pivoting, or the regular LU decomposition. pivot= False not supported on CPU. Default: True.

get_infos (bool, optional) – if set to True, returns an info IntTensor. Default: False

out (tuple, optional) – optional output tuple. If get_infos is True, then the elements in the tuple are Tensor, IntTensor, and IntTensor. If get_infos is False, then the elements in the tuple are Tensor, IntTensor. Default: None

A tuple of tensors containing

factorization (Tensor): the factorization of size $(*, m, n)(*, m, n)(*, m, n)$

pivots (IntTensor): the pivots of size $(*, \min(m, n))(*, \min(m, n))(*, \min(m, n))$. pivots stores all the intermediate transpositions of rows. The final permutation perm could be reconstructed by applying `swap(perm[i], perm[pivots[i] - 1])` for $i = 0, \dots, \text{pivots.size}(-1) - 1$, where perm is initially the identity permutation of mmm elements (essentially this is

what `torch.lu_unpack()` is doing).

`infos` (`IntTensor`, optional): if `get_infos` is `True`, this is a tensor of size $(*)^{(*)} (*)$ where non-zero values indicate whether factorization for the matrix or each minibatch has succeeded or failed

(`Tensor`, `IntTensor`, `IntTensor` (optional))